El profesor ha subido la **plantilla ERS** que es el documento que tenéis que rellenar y el **PowerPoint** resume el enunciado en 30 diapositivas (no trae nada nuevo, solo detalles en las notas del orador).

1. El enunciado

El reto. Tenéis que resolver una necesidad pequeña con una mini web que consulte una API de IA. No se valora añadir funciones, sino **hacer y explicar bien el proceso completo**: análisis, desarrollo, pruebas, Git, despliegue y defensa.

Son dos aplicaciones, no una:

	App 1	App 2
Cómo se hace	A mano, poco a poco, entendiendo cada línea	Desde cero, con **GitHub Spec Kit** y asistencia de IA
Funcionalidad	La función elegida	La misma, más autenticación con **Clerk**
Despliegue	Front y Back en **Vercel**	Primero en **Vercel**, luego con **Docker en AWS EC2**

Al final hay que **comparar los dos procesos**: tiempo, calidad, errores, mantenibilidad y en qué ayudó o falló la IA.

La regla de simplicidad es obligatoria y no admite excepciones: una pantalla, un formulario, una función útil y una sola API de IA. El flujo tiene que ser exactamente este:

1. El usuario rellena el formulario de la SPA.
2. El Front envía la petición a FastAPI.
3. FastAPI valida los datos y hace **una única llamada** a la API de IA.
4. FastAPI devuelve JSON y el Front lo muestra en la misma pantalla.

Está prohibido: cualquier base de datos o persistencia (incluidas cachés, ficheros, `localStorage` y `sessionStorage`), CRUD, paneles, varias pantallas o rutas (no hace falta router), gestión de usuarios propia, chat con historial, subida de archivos, RAG, colas y **"búsquedas adicionales o servicios extra"**. Si añadís algo de esto no suma puntos: hay que quitarlo antes de entregar.

Tecnologías obligatorias: Python + FastAPI, pytest, Vue o React, **Tailwind CSS**, Git/GitHub, Vercel, Spec Kit, Clerk, Docker y EC2. El proveedor de IA lo elegís vosotros (OpenRouter sirve).

Contrato y errores. Un endpoint del tipo `POST /api/procesar` que devuelva `{"resultado": "..."}` cuando todo va bien y un JSON de error controlado cuando falla. Hay que documentar los códigos: 200 éxito, 422 validación, 502 fallo del proveedor, 504 timeout y 401 sin token (solo App 2). También se exige un límite de longitud de entrada y un timeout. La respuesta de la IA se muestra como texto, nunca se ejecuta como HTML.

Pruebas con pytest y sin llamar a la IA real. Se simula el proveedor con mocks. Los casos mínimos son: entrada válida (con una sola llamada al mock), vacía o solo espacios, en el límite de longitud y por encima, fallo del proveedor, timeout y respuesta inválida.

Git. Un repositorio por grupo, con contribuciones identificables de cada uno y una evolución visible por hitos (H0 a H12: inicio, backend mínimo, contrato, tests, frontend…). Un único commit al final no vale. En App 2, **la especificación tiene que aparecer en Git antes que el código**.

Seguridad. La clave de IA vive solo en el Backend, nunca en una variable `VITE_*`. Se suben `.env.example` y `.gitignore`, y en Docker también `.dockerignore`. Si se filtra un secreto, hay que revocarlo, porque borrarlo del último commit no lo saca del historial.

Entregables: el repositorio, `docs/ERS.md` (y la ERS de la App 2), diagramas Mermaid, pruebas, los artefactos de Spec Kit, `docs/uso-ia.md`, la comparativa, las URLs de despliegue, el README y **un PowerPoint propio del grupo**.

Defensa: de 8 a 10 minutos más preguntas, con participación de todos. La demo debe mostrar una entrada válida, un error controlado, el acceso con Clerk y el rechazo sin token, la App 2 funcionando en EC2, y un test con mock junto a un commit para justificar el proceso.

Rúbrica:

Criterio	Peso
ERS y análisis	15 %
Aplicación 1	20 %
Calidad técnica, tests y Git	15 %
App 2, Spec Kit e IA	15 %
Vercel, Clerk, Docker y AWS	15 %
Documentación	10 %
Exposición y defensa	10 %

2. La plantilla ERS

Es la Especificación de Requisitos del Software que tenéis que entregar en `docs/ERS.md`, **manteniendo su estructura**. Sus apartados son: datos del documento, introducción (objetivos y alcance), descripción del sistema, requisitos funcionales con fichas y criterios “Dado / cuando / entonces”, requisitos no funcionales medibles, datos, interfaces, casos de uso, trazabilidad, cuestiones pendientes y revisión.

Lo importante para rellenarla:

- **Entre 3 y 6 requisitos funcionales y entre 2 y 4 no funcionales**, cada uno comprobable. “La app será rápida” no vale.
- Donde algo no aplique se pone **“No aplica: [motivo]”**, sin inventar contenido para llenar.
- Hay que borrar los ejemplos de la plantilla (reservas, administradores…).
- El apartado 5.1 sí aplica aunque no haya base de datos: describe los datos **transitorios** de entrada y salida.

3. El PowerPoint

Resume el enunciado diapositiva a diapositiva. Os sirve como guía rápida.

4. Lo que cambia respecto a lo que hablamos

El enunciado actual modifica parte de lo que tenía apuntado como notas del profesor:

a) Vercel sí es obligatorio. Propuse quitar Vercel por que creía que el profesor querría el despliegue en AWS. En realidad hay que desplegar **las dos apps en Vercel** y además **la App 2 en EC2 con Docker**. La sección de despliegue de su documento tampoco vale tal cual: plantea Vercel Functions con Node y Upstash, y aquí sería FastAPI en Vercel, sin caché.

b) El planificador de itinerarios, tal como está planteado, incumple el enunciado. La llamada a Overpass/OpenStreetMap es justo una "búsqueda adicional" para enriquecer el resultado, y eso está prohibido: solo se permite una API, la de IA. Además, "Mis planes", el guardado en `localStorage`, la caché, el router con varias vistas y OSRM también están prohibidos. Hay dos salidas:

- **Mantener la idea pero solo con IA**: el modelo genera el itinerario a partir del formulario, con un aviso de que los sitios y horarios hay que comprobarlos. Se pierde la "lista cerrada", que era lo mejor de la idea, pero cumple las normas.
- **Elegir una idea más sencilla**, del estilo de las que propone el enunciado (resumidor, corrector, generador de preguntas…).

Yo consultaría al profesor si un planificador solo con IA le parece bien, porque el enunciado pide revisar el alcance con él en la fase A.

c) Choque con lo de imprimir en la terminal. Me dijiste que el profesor espera ver en la consola la petición y la respuesta. Pero el enunciado dice que **"los registros técnicos no deben incluir textos, respuestas, tokens ni secretos"**. Las dos cosas se contradicen, así que preguntadle. Una solución intermedia sería registrar solo metadatos (hora, campos recibidos, longitud, código de respuesta, duración) y mostrar el contenido completo únicamente en desarrollo local, activándolo con una variable de entorno.

Con esto sigo teniendo dudas, creo que lo mejor es olvidar lo de imprimir la información en terminal.

d) Hay muchos más requisitos de los que teníamos: pytest con mocks, Tailwind, Spec Kit, Clerk, Docker, la ERS formal, la comparativa y el PowerPoint propio. El trabajo es bastante más grande de lo que parecía, y la parte de código es la menor.